

Shuffling for Safeguarding (SfS): Replay Protection in Wireless Broadcast Systems

1st Ala Darabseh

New York University Abu Dhabi
Abu Dhabi, United Arab Emirates
ala.darabseh@nyu.edu

2nd Christina Pöpper

New York University Abu Dhabi
Abu Dhabi, United Arab Emirates
christina.poepper@nyu.edu

Abstract—Replay attacks pose a significant threat to wireless communication systems by reusing valid messages at inappropriate times, leading to compromised data integrity and security risks. This problem is particularly critical in safety- and time-sensitive domains such as drone remote identification, vehicular safety beacons, and aviation broadcasts, where freshness must be ensured under strict timing constraints. Existing defenses typically bind freshness through explicit timestamps or sequence numbers protected by cryptographic authentication, which adds extra fields or additional headers. In this paper, we propose *Shuffling for Safeguarding (SfS)*, a replay protection scheme that detects and prevents replay attacks for signed messages by providing *implicit freshness* without adding message overhead. SfS does not introduce explicit time fields; rather, it transmits payload bits by shuffling them according to a time-derived permutation uniquely determined by the message’s generation epoch. At the receiver, verification jointly checks the signature and the consistency of the observed permutation with the expected transmission epoch, enabling detection of replayed messages. We validate our approach on a Raspberry Pi Wi-Fi ad-hoc setup (IEEE 802.11, 2.4 GHz) using UDP/IP traffic, evaluating delays down to the millisecond scale under broadcast loss and clock skew. Experimental results demonstrate near-perfect rejection of cross-epoch replays, even at challenging millisecond-level delays, highlighting the effectiveness and practicality of our approach for signed wireless applications.

Index Terms—Wireless Transmissions, Replay Attacks, Detection and Prevention, Message Integrity

1. Introduction

Remote ID systems require unmanned aerial vehicles (UAVs) to periodically broadcast identity and operational information over open wireless channels such as Bluetooth and Wi-Fi [1]. These broadcasts support regulatory compliance, airspace awareness, and local identification of nearby drones. As authenticated Drone Remote ID deployments continue to emerge, replay protection becomes increasingly important. A previously valid broadcast can be captured and retransmitted later to create phantom drones, false restricted-

zone violations, misleading airspace activity, or stale location information despite carrying a valid cryptographic signature [2], [3]. In such systems, authenticity alone is insufficient if message freshness cannot also be guaranteed.

The same freshness challenge appears in many wireless broadcast systems where information is highly time-sensitive. Examples include ADS-B aviation surveillance [4], vehicular safety beacons (V2X) [5], satellite navigation broadcasts [6], and industrial wireless telemetry systems, where stale yet authentic packets may mislead tracking, localization, collision-avoidance, or operational decision logic [2], [3], [7], [8]. In these environments, even modest delays, ranging from milliseconds to a few seconds depending on the application, may transform an otherwise legitimate message into a harmful one [7], [8].

These systems currently exhibit different levels of replay resilience maturity. Emerging authenticated Drone Remote ID systems often lack strong replay protection despite increasing reliance on signed broadcasts. Vehicular safety systems (V2X) already employ timestamp-based freshness validation, but at the cost of additional metadata overhead and synchronization requirements. In contrast, several proposed authenticated ADS-B and satellite telemetry architectures focus primarily on authenticity and integrity, while replay resilience remains weak or absent. SfS is particularly attractive in such constrained broadcast environments because it enables freshness verification without requiring additional timestamp or counter fields.

Anti-replay protection is commonly achieved using explicit freshness metadata such as timestamps, sequence numbers, counters, or challenge-response exchanges protected by cryptographic authentication mechanisms (MACs/signatures) [9]–[11]. While effective, these approaches introduce several practical limitations. Explicit freshness fields consume valuable payload space in constrained broadcast protocols, require replay-window state management at receivers, and often depend on synchronization assumptions that may be difficult to maintain in lightweight wireless systems or legacy protocol deployments. Alternative defenses may additionally rely on external infrastructure, specialized hardware, or interactive communication that introduces latency and operational complexity [12]–[14].

This challenge is particularly important for wireless broadcast systems such as Drone Remote ID, where packets are periodically transmitted over bandwidth-constrained channels and must remain lightweight, low-latency, and backward compatible. In such settings, introducing additional freshness fields or protocol modifications may be undesirable or impractical.

In this paper, we introduce a lightweight replay *detect-and-prevent* mechanism that operates entirely within existing packet formats. Our approach—*Shuffling for Safeguarding (SfS)*—binds freshness into what is already authenticated. Instead of transmitting explicit timestamps or counters, the sender applies a *deterministic, epoch-derived bit-order shuffling* to the payload before signing the message. Upon reception, the receiver reconstructs the expected permutation(s) for the current acceptance window and verifies the signature accordingly. Legitimate in-window packets verify successfully and are accepted, while stale replayed packets fail verification because their payload permutation no longer matches the expected transmission epoch.

SfS achieves replay resistance with *zero additional payload overhead*, no extra round-trip latency, and minimal synchronization requirements. The approach is particularly attractive for constrained wireless broadcast systems such as Drone Remote ID, while also generalizing to broader applications including ADS-B aviation broadcasts, vehicular safety beacons, satellite telemetry, and other time-sensitive wireless communication systems where freshness is critical.

1.1. Problem Statement

Many wireless broadcast systems are vulnerable to *replay attacks*, where a previously valid message is captured and retransmitted after a delay [23], [24]. Although digital signatures and MACs provide message integrity and authenticity, they do not inherently guarantee *freshness* [25]. Consequently, stale yet authentic packets may still be accepted as valid.

In time-sensitive broadcast systems, replay delays ranging from milliseconds to seconds may already produce harmful effects. Table 1 summarizes representative timing constraints across several wireless broadcast applications. A common approach to freshness enforcement is to include explicit *timestamps* or *sequence numbers* protected by cryptographic authentication. However, these approaches (i) consume scarce payload bits in constrained broadcast protocols, (ii) require replay-window state management and timer synchronization, and (iii) may be impractical when packet formats are fixed or already fully utilized.

Problem. We seek a receiver-side replay *detect-and-prevent* mechanism that: (i) is compatible with existing signature or MAC-based authentication schemes (ii) avoids extra round trips, (iii) *does not add explicit freshness fields* (timestamps/counters) to the packet, and (iv) remains robust to modest loss and clock skew.

1.2. Contributions

This paper presents *Shuffling for Safeguarding (SfS)*, a lightweight replay detect-and-prevent mechanism for signed wireless broadcasts. The main contributions are :

- 1) **A lightweight replay-resilient freshness mechanism for broadcast systems.** We introduce SfS, a payload-permutation mechanism that implicitly binds transmission time to authenticated message structure, enabling replay detection without explicit freshness fields such as timestamps or counters. SfS is compatible with existing authentication schemes (e.g., RSA, ECDSA, Ed25519, HMAC) and is suitable for constrained wireless broadcast systems such as Drone Remote ID and ADS-B.
- 2) **Application-grounded evaluation.** Using Drone Remote ID as the primary motivating case study, we further demonstrate how SfS generalizes to ADS-B, vehicular safety beacons, and satellite broadcast systems.
- 3) **Real-world experimental validation.** We implement SfS on a Raspberry Pi Wi-Fi ad-hoc testbed and evaluate replay attacks at both second- and millisecond-scale delays, demonstrating strong replay detection performance with minimal overhead.

2. Background and Related Work

Replay attacks reuse previously valid transmissions outside their intended freshness period while preserving correct formatting and cryptographic validity. Such attacks are particularly problematic in wireless broadcast systems, including Drone Remote ID, ADS-B, and vehicular communication, where messages are transmitted under strict latency and payload constraints. Existing replay defenses generally rely on three main directions: (A) explicit freshness information, (B) physical-layer or contextual consistency checks, and (C) data-driven anomaly detection.

A. Explicit Freshness Mechanisms

(A1) Timestamp-based freshness. The most direct replay defense is to embed timestamps within authenticated messages. Similar approaches are widely used in wireless and IoT systems because they are simple to verify and integrate naturally with existing authentication mechanisms [26]. However, in practice, such deployments consume additional payload space and require bounded timing synchronization together with careful handling of jitter, clock drift, and retransmissions.

(A2) Nonces, counters, and challenge-response. To avoid dependence on explicit timestamp synchronization, many protocols bind a one-time nonce or a monotonically increasing counter into the authenticated data; receivers maintain a replay window/cache. This approach prevents duplicates without requiring tight time synchronization, but still admits *delayed* messages if they remain inside the window. Challenge-response mechanisms strengthen freshness guarantees by proving liveness, but introduce additional communication rounds and latency costs, particularly problematic for lossy wireless links.

TABLE 1: Acceptable delays and replay risks in broadcast location-based applications. Systems with frequent updates tolerate only short replay delays, whereas slower-update systems tolerate larger delays. The tolerances have been estimated by assessing the transmission intervals of location data

System/Application	Use Case	Example Technologies	Acceptable Delays
Aviation [12], [15]	Aircraft broadcast real-time position/velocity	ADS-B (Automatic Dependent Surveillance–Broadcast)	milliseconds to second
Vehicular Networks [16]	Vehicles broadcast position, speed, intent	DSRC, C-V2X, VANET, IEEE 802.11p	milliseconds to second
Satellite Navigation [17], [18]	GNSS satellites broadcast navigation/time data	GPS, Galileo, GLONASS, BeiDou	milliseconds
Marine Communication [19], [20]	Ships broadcast position, course, speed	AIS (Automatic Identification System)	seconds
GPS Fleet/Asset Tracking [21], [22]	Vehicles/containers broadcast GPS location	GPS trackers, ALTS, RTLS	seconds

Several deployed wireless protocols illustrate these trade-offs in practice. Formal analyses of WPA2 showed that replayed authentication exchanges could trigger nonce reinstallation and keystream reuse despite otherwise valid cryptographic authentication [27]. Similarly, replay resilience in cellular control-plane protocols often relies on timestamp caches and replay-window management, introducing additional synchronization and storage overheads [28]. Replay protection improvements proposed for constrained IoT systems, such as OSCORE-NG, further demonstrate the need for tighter nonce and counter management while preserving lightweight operation [29].

(A3) Delayed key disclosure (TESLA family). TESLA-based protocols authenticate broadcast traffic by disclosing authentication keys after controlled delays, enabling one-way authenticated communication with loose time synchronization [30]. Correctness depends on “receipt safety” where receivers must ensure packets were received before corresponding keys were disclosed [31]. Recent work formalizes synchronization requirements and practical receiver designs for GNSS-like environments [32]. Although TESLA reduces communication overhead compared with challenge–response mechanisms, it remains dependent on synchronization quality and delayed key schedules.

Takeaway. While widely deployed, these mechanisms introduce synchronization assumptions, replay-window management, receiver-side state, or additional communication overhead that may become problematic in constrained broadcast environments. In practice, incorrect state handling can also re-open replay vulnerabilities despite otherwise strong cryptographic protection [33]. These limitations motivate lightweight replay defenses that preserve one-way communication while minimizing protocol overhead.

B. Physical/Context-Based Replay Detection

When cryptographic metadata is insufficient or costly, replay detection may instead rely on validating whether a received signal is physically consistent with the current time and propagation environment.

(B1) PHY/Rf fingerprinting. Hardware imperfections and analog transmission characteristics can act as device fingerprints, allowing replayed or relayed signals to be distinguished from legitimate live transmissions. Prior surveys

and experimental studies demonstrate that RF fingerprinting can support replay detection across wireless environments, including satellite communication systems [34]. However, performance may degrade under mobility, hardware variation, environmental change, or calibration drift.

(B2) Channel and propagation consistency. Beyond hardware-specific signatures, replay detection can also exploit Channel State Information (CSI) to verify propagation consistency. Since CSI is affected by the surrounding wireless environment, it decorrelates over time and space in dynamic settings. As a result, a replayed transmission may exhibit subcarrier amplitude and phase measurements that align more closely with previously observed CSI profiles than with the receiver’s current channel conditions, allowing replay behavior to be inferred [35]. Other approaches use propagation delay, distance bounding, or OFDM channel fingerprints to identify relay and replay behavior [36]. Although effective, such approaches often require PHY-layer access, specialized hardware, or careful environmental calibration.

Takeaway. Physical-layer replay defenses provide strong liveness properties tied to the transmission environment itself. However, they often require PHY-layer access, specialized sensing infrastructure, additional calibration, or environment-specific assumptions that may limit deployment in lightweight wireless systems.

C. Learning-Based Replay Attack Detection

(C1) ML anomaly detection and traffic semantics. Data-driven replay detection approaches leverage traffic timing, sequence behavior, topology evolution, or message semantics to identify replay anomalies. In vehicular networks, supervised learning models have been proposed to detect replay and replay-Sybil attacks with high accuracy under controlled conditions [2]. In aviation systems, Transformer-based sequence models detect ADS-B replay attacks directly from broadcast message streams [3]. Other ADS-B replay defenses compare incoming broadcasts against geographically distributed sensor observations or previously observed traffic patterns [37], [38]. Although these techniques provide protocol-agnostic replay detection, they often require extensive training data, dense sensing infrastructure, or computationally intensive real-time analysis while remaining sensitive to distribution shift and adversarial adaptation.

Motivation and Positioning of SfS. Existing replay defenses expose recurring trade-offs between freshness guarantees, synchronization requirements, protocol overhead, infrastructure dependence, and verification latency. These trade-offs become particularly challenging in constrained wireless broadcast systems such as ADS-B and Drone Remote ID, where packet formats, payload sizes, and timing constraints are tightly restricted. SfS addresses this problem differently by binding transmission time directly to the authenticated payload structure through time-derived bit permutations. Unlike timestamp or counter-based schemes, SfS does not require explicit freshness metadata inside the transmitted packet, while still relying on bounded timing consistency between sender and receiver. At the same time, it avoids the infrastructure and calibration requirements associated with PHY-layer replay detection while preserving one-way verification suitable for broadcast communication.

3. Threat Model and Attack Scenarios

An adversary $\mathcal{A}$ operates between sender S and receiver R over an insecure wireless channel. The adversary can eavesdrop, delay, reorder, relay, and replay packets, including fast relays with negligible processing delay. Captured packets may be re-injected after arbitrary delays Δt , ranging from milliseconds to seconds. We consider the following **attack scenarios**:

- 1) **Classic delayed replay** ($\Delta t > W$): $\mathcal{A}$ captures valid packets and replays them outside the receiver acceptance window W , referred to as the Threshold Value (THV). *Outcome*: The replayed packet is rejected because it no longer satisfies the receiver freshness-validation condition, resulting in signature/MAC verification failure.
- 2) **Fast relay / within-window replay** ($\Delta t \leq W$): $\mathcal{A}$ re-injects captured packets within the receiver acceptance window, often with near-zero delay. *Outcome*: The cryptographic verification succeeds because the replay remains within the same epoch; duplicate suppression may additionally be enforced using a replay cache.
- 3) **Permutation-aware replay**: $\mathcal{A}$ captures a packet and attempts to reorder its bits to match the expected permutation at replay time. *Outcome*: Signature verification fails because the authenticated digest commits to the original permutation order, and the adversary cannot re-sign the modified payload.

Attack Model Assumptions. Sender and receiver cryptographic keys remain uncompromised; $\mathcal{A}$ cannot forge signatures, invert secure hashes, or derive secret keys; any payload reordering changes the signed payload structure and therefore invalidates the original signature.

4. SfS Design and Security Rationale

SfS implicitly binds transmission time to authenticated payload structure without introducing additional protocol fields. Instead of transmitting explicit freshness metadata,

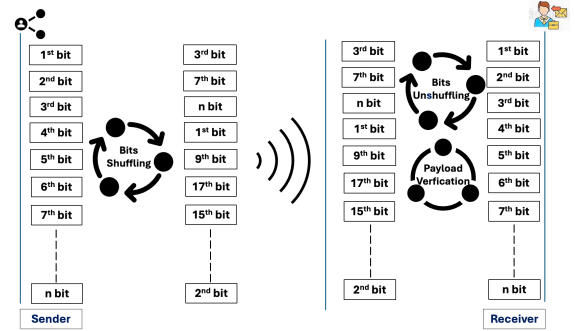


Figure 1: Overview of the Proposed "Shuffling for Safe-guarding (SfS)" System—Bits Shuffling at Sender and Unshuffling for Payload Verification at Receiver

the sender applies a time-derived permutation to the payload before authentication. The receiver reconstructs the expected permutation(s) within the acceptance window and verifies the received message accordingly. Figure 1 summarizes the overall workflow.

4.1. System Assumptions

We assume that: (i) messages are already signed or authenticated within the underlying protocol, (ii) cryptographic keys are securely established beforehand, (iii) sender and receiver maintain sufficiently synchronized clocks through existing synchronization mechanisms (e.g., NTP/PTP), and (iv) unmodified protocol fields required for system operation (e.g., sender location fields) remain transmitted normally.

4.2. Time-Derived Seed Generation

SfS derives a deterministic seed from the message transmission time, which is used to generate the payload permutation. Let t denote the sending time (e.g., in seconds or milliseconds). The seed is computed as:

$$s = f(t) \quad (1)$$

where $f(\cdot)$ is a deterministic mapping from time to an integer seed. This ensures that both sender and receiver can independently reconstruct the same seed using their local time estimation. The seed is then used to initialize a pseudo-random permutation generator:

$$P(t) = \text{Perm}(s) \quad (2)$$

where $P(t)$ denotes the permutation applied at time t . Because the mapping is deterministic, the receiver can regenerate candidate permutations within a bounded time window and verify consistency.

This design enables synchronization without requiring additional metadata transmission, while binding the permutation to the transmission epoch.

4.3. Sender-Side Procedure

The sender begins by obtaining the current time and deriving a seed uniquely associated with this timestamp. Each bit-shuffling sequence reflects the message creation time, allowing the sender and receiver to infer timing information without an explicit timestamp. Using this seed, a deterministic permutation over the n payload indices is generated, where n denotes the payload length. The sender then shuffles the payload bits according to this permutation.

To ensure integrity and authenticity, the permutation is combined with the message content during hashing, and the resulting hash is signed using a standard cryptographic scheme (e.g., SHA-256 [39] with a digital signature). The sender then transmits the payload bits in the permuted order defined by the time-derived sequence.

- 1) **Time-Based Seed and Permutation Generation:** The sender begins by obtaining the current transmission time t_{current} and deriving the corresponding seed and permutation according to (1) and (2). Let

$$Q = P(t_{\text{current}}) \quad (3)$$

denote the resulting permutation over the payload index set $\{1, 2, \dots, n\}$, where n is the payload length. The payload bits are then rearranged according to Q prior to hashing and signing.

- 2) **Message Signing and Broadcast:** The sender applies the time-derived permutation Q to the payload M , producing the permuted message M_Q . A hash $H = \text{SHA256}(M_Q)$ is then computed and signed using the sender's private key. Finally, the sender broadcasts $\{M_Q, \text{Signature}\}$. The permutation Q is not transmitted explicitly; instead, it is implicitly determined by the time-derived seed and reconstructed by the receiver.

4.4. Receiver-Side Procedure

Upon receiving the permuted message and its corresponding signature, the receiver reconstructs the expected permutation from the estimated transmission epoch and verifies message integrity.

- 1) **Extract Sender Information:** The receiver extracts the sender's location information from the received message M_Q .
- 2) **Estimate Sending Time:** Using the sender and receiver locations, the receiver estimates the propagation delay as $\Delta t = \frac{d}{c} + t_{\text{delay}}$, where d is the geometric distance between sender and receiver, c is the speed of light, and t_{delay} accounts for additional processing and network delays. The transmission time is then approximated as $t_{\text{send}} = t_{\text{receive}} - \Delta t$.
- 3) **Reconstruct Permutation:** Using t_{send} , the receiver regenerates the expected permutation Q_{expected} according to Eq. (1) and (2).
- 4) **Unshuffle Payload:** The receiver applies the inverse permutation to the received payload M_Q to recover the original message M .

TABLE 2: Observed permutation recurrence over one week

Payload (L)	Permutation Space ($L!$)	Observed Repetitions	Observation
6	720	Frequent	High reuse, predictable
10	$\approx 3.6 \times 10^6$	Moderate	Rare, mostly pairwise
14	$\approx 8.7 \times 10^{10}$	Very few	Sparse, long gaps
22	$\approx 10^{21}$	None	No recurrence observed
112	Extremely large	None	No recurrence observed

- 5) **Verify Signature:** The receiver computes $H_{\text{expected}} = \text{SHA256}(M_Q)$ and verifies the received signature using the sender's public key.
- 6) **Validate Message:** If signature verification succeeds, the message is accepted; otherwise, it is rejected.

Appendix A provides a detailed sequence diagram summarizing the sender- and receiver-side operations of Sfs.

4.5. Permutation Recurrence & Replay Feasibility

In Sfs, each message is transformed using a permutation derived from a time-dependent seed. Since the permutation space is finite, permutation reuse may theoretically occur over time. This section evaluates the likelihood of such recurrence and its implications for replay attacks.

4.5.1. Permutation Space and Recurrence Analysis. For a payload of length L , the number of possible permutations is $N = L!$. Although finite, the permutation space grows factorially with payload length, becoming extremely large even for moderate payload sizes. For n transmitted messages, the expected number of permutation collisions is approximated by $(E \approx \frac{n(n-1)}{2L!})$, indicating that recurrence rapidly becomes negligible as L increases.

To evaluate this behavior empirically, we generated time-derived permutations over one week of operation at second-level resolution ($n \approx 604,800$ messages). Table 2 summarizes the observed recurrence behavior for different payload lengths. The results show a clear transition as the payload length increases. For small payloads, recurrence becomes more likely due to the limited permutation space, whereas for realistic payload sizes ($L \geq 22$), no recurrence was observed during the evaluation period.

4.5.2. Replay Feasibility and Security Implications. Permutation recurrence alone does not enable replay attacks. Successful replay additionally requires both $P(t_2) = P(t_1)$ and $|t_2 - \hat{t}_s| \leq W$, where $\hat{t}_s$ denotes the receiver-estimated sending time and W is the receiver acceptance window.

Consequently, an adversary must not only wait for permutation reuse, but also replay the message within the receiver's accepted timing window. As payload length increases, both recurrence and successful temporal alignment become increasingly impractical, rendering replay attacks based on permutation recurrence infeasible under realistic operating conditions.

4.6. Mathematical Model and Security Analysis

Let $m \in M$ denote a payload message and let $P(t)$ denote the deterministic payload permutation generated from

the time-dependent seed construction described in Section 4.2. A sender transmitting at time t_1 computes the permuted payload $m_Q = \text{Permute}(m, P(t_1))$, computes its hash $h = H(m_Q)$, and generates a signature or MAC σ over the authenticated payload representation.

Upon reception, the receiver reconstructs the expected permutation from the estimated transmission epoch, as described in Section 4.4, and verifies the received authenticated payload representation using the corresponding verification procedure.

4.6.1. Model Properties. Assume $H : M \rightarrow \mathcal{H}$ is a collision-resistant and preimage-resistant hash function. Each entity X owns a public/private key pair (PK_X, SK_X) and employs a secure authentication scheme (Sign, Verify).

The sender transmits the authenticated pair (m_Q, σ) , while the receiver accepts the message only if verification succeeds under the reconstructed permutation corresponding to the estimated transmission epoch.

4.6.2. Security Definitions. Definition 1 (Integrity). The system is resistant to tampering if any adversarial modification $m'_Q \neq m_Q$ causes signature or MAC verification failure except with negligible probability.

Definition 2 (Authenticity and Non-Repudiation). Only the legitimate sender holding SK_S can generate a valid authenticated payload representation for m_Q .

Definition 3 (Replay Resistance). A replayed message transmitted at time t_1 and replayed at time t_2 succeeds only if

$$P(t_2) = P(t_1) \quad \text{and} \quad |t_2 - \hat{t}_s| \leq W \quad (4)$$

where $\hat{t}_s$ denotes the receiver-estimated sending time and W is the receiver acceptance window.

4.6.3. Security Proofs. Lemma 1 (Integrity). Any modification of m_Q invalidates the authenticated payload representation.

Proof Sketch. Any payload modification changes $H(m_Q)$. Due to collision resistance, the original signature or MAC no longer verifies successfully.

Lemma 2 (Authenticity and Non-Repudiation). Only the legitimate sender can generate a valid authenticated payload representation.

Proof Sketch. Under standard unforgeability assumptions, generating a valid signature or MAC without knowledge of the sender's secret key is computationally infeasible.

Lemma 3 (Replay Resistance). A replayed packet is rejected whenever the receiver reconstructs a different permutation from the original transmission permutation.

Proof Sketch. The authenticated payload representation is generated using the permutation associated with the original transmission epoch $P(t_1)$. During replay verification, the receiver reconstructs the expected permutation from the estimated sending time. If $P(t_2) \neq P(t_1)$, the reconstructed payload ordering becomes inconsistent with the authenticated payload representation, causing verification failure.

Consequently, successful replay requires simultaneous permutation recurrence and temporal alignment with the receiver acceptance window. As discussed in Section 4.5, this becomes increasingly impractical for realistic payload sizes due to the rapid growth of the permutation space.

An extended formalization, including additional definitions and detailed proofs, is provided in Appendix B.

5. Results and Evaluation

5.1. Experimental Setup

We evaluated the proposed Sfs transmission approach using a real-world experimental testbed. Specifically, we deployed a VANET setup consisting of three Raspberry Pi Model B devices [40] configured in an IEEE 802.11 Ad-Hoc (IBSS) network [41]. Experiments were conducted in an indoor laboratory environment over the 2.4 GHz band under typical background Wi-Fi activity.

The devices were placed within line-of-sight at distances of approximately 3–5 meters and remained static throughout the experiments. In this configuration, the sender broadcasts packets that can be received by all nodes within communication range. Each message was processed using the Sfs mechanism, where payload bits are permuted based on a time-derived seed prior to transmission. The permuted payload is then hashed and signed using RSA digital signatures with SHA-256 [42]. Each node is provisioned with the sender's public key to enable signature verification. Packet exchange was implemented in Python 3 using UDP sockets, providing lightweight and connectionless communication. Each Raspberry Pi operates on a Linux-based system and is equipped with a built-in 2.4 GHz Wi-Fi adapter configured on Channel 6 (2.437 GHz). The transmission power was set up to 31 dBm.

To ensure consistent time synchronization across nodes, we used Chrony [43], a lightweight implementation of the Network Time Protocol (NTP). One device acted as a local time server, while the others synchronized as clients, enabling reliable timing coordination within the isolated Ad-Hoc network. This setup allows us to evaluate the correctness and reproducibility of the time dependent permutation mechanism under realistic wireless conditions.

5.2. Evaluation Metrics

To assess the practicality and accuracy of the proposed Sfs approach, we consider the following evaluation metrics:

- **System Correctness (No-Attack Applicability):** We evaluate whether the time-dependent shuffling mechanism operates correctly under normal conditions (i.e., in the absence of adversarial interference). *Question:* Does the time-derived shuffling of payload bits, combined with digital signatures, preserve correctness for legitimate traffic? *Method:* The sender broadcasts signed packets where payload bits are shuffled according to a time-derived permutation. The receiver reconstructs the expected shuffling

within a configured time window W (THV) and verifies the received messages. *Metrics*: Legitimate acceptance rate (True Positive Rate, TPR) and False Negative Rate (FNR) as a function of the time window W .

- **Detection Performance Under Replay**: We evaluate the accuracy of our approach under various attack models, measuring how many replayed messages were successfully detected across different time windows and delay intervals. This evaluation captures the sensitivity of SfS to timing deviations and its ability to reject temporally inconsistent replays. *Question*: Do the proposed replay attacks (Δt at second- and millisecond-scales) *fail cryptographic verification* as designed due to mismatch in the time-derived shuffling when $\Delta t > W$? *Method*: Perform eavesdrop-and-replay attacks with controlled delays at second and millisecond scales, while sweeping the acceptance window W (THV). *Metrics*: TPR/TNR, Precision/Recall/F1; ROC/AUC per delay; decision frontier over $(\Delta t, W)$.
- **Overhead and Throughput (Practicality)**: We assess the computational and communication overhead introduced by SfS, focusing on the cost of permutation-based shuffling and signature operations. *Question*: What is the computational cost on embedded hardware, and does SfS add on-wire overhead? *Method*: Measure per-packet sender time (seed/sequence, shuffle, hash, sign) and receiver time (reconstruct, unshuffle, hash, verify) on Raspberry Pi 3B/3B+; stress with back-to-back bursts. *Metrics*: Signing and verification time (μ s), sustainable throughput in packets per second (pkts/s), end-to-end latency; on-wire bytes (no extra fields).

5.3. Results

Figure 2 shows a roadmap of the evaluation structure and the corresponding figures/tables used throughout this section.

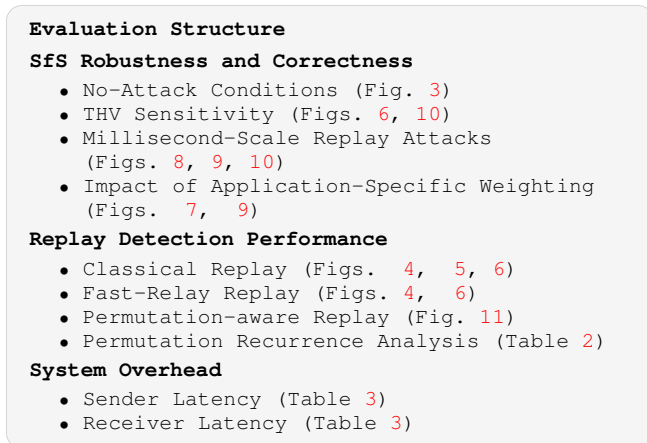


Figure 2: Evaluation structure and corresponding figures/tables.

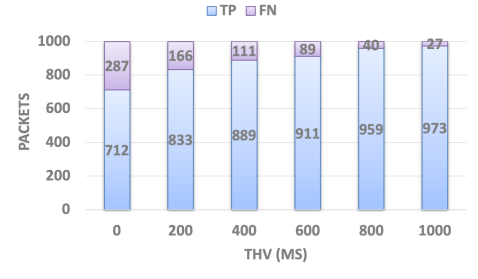


Figure 3: System correctness under no-attack conditions: TP and FN versus THV. Larger THVs reduce FN by increasing timing tolerance.

5.3.1. System Correctness (No-Attack Applicability).

To evaluate the feasibility of embedding the sending time through shuffled payload bits within each packet, we conducted a baseline experiment under ideal conditions without any adversarial interference. The objective was to determine whether the receiver could correctly unshuffle the payload, reconstruct the expected permutation from the estimated sending time, and validate the packet accordingly.

A total of 1000 packets were transmitted between the sender and receiver in each experiment. For every packet, we evaluated whether it was accepted or rejected by the receiver. Packet acceptance depended on the receiver generating a permutation consistent with the one used by the sender during transmission. To account for real-world factors such as propagation delay and timing variations, we introduced a tolerance window defined by a predefined *Threshold Value* (THV), expressed in milliseconds. A packet was considered valid if the estimated sending time fell within a $\pm THV$ range around the expected timestamp.

Figure 3 illustrates the effect of varying THV on system correctness under no-attack conditions. As THV increases, the receiver becomes more tolerant to timing variations, leading to a reduction in false negatives and improved acceptance of legitimate packets. However, enlarging the acceptance window introduces a trade-off. A larger THV increases the likelihood that replayed packets, especially those with small delays (e.g., millisecond-scale replays), fall within the acceptable range. This highlights the inherent balance between robustness to timing noise and sensitivity to replay attacks.

5.3.2. Detection Performance Under Replay. After establishing the system’s performance under normal (attack-free) conditions, we evaluated the ability of SfS to detect replay attacks under different delay scenarios derived from our threat model.

Classical and Fast-Relay Replays First, we evaluate SfS under both classic delayed replay (outside-window) and fast-relay (within-window) conditions. For each configured acceptance window W (THV), we conducted five replay trials in which an adversary re-injected previously captured packets after controlled delays $\Delta t \in \{1, 2, 3, 4, 5\}$ seconds.

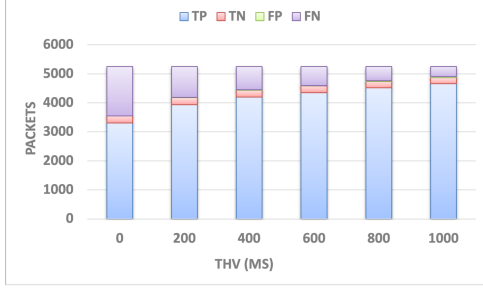


Figure 4: Trade-off between legitimate-packet acceptance and replay-detection performance across all replay experiments and receiver acceptance-window values.

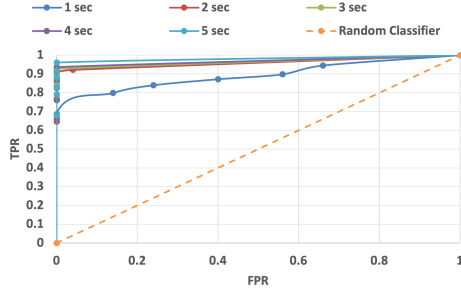


Figure 5: ROC curves for replay attacks delayed by 1–5 s. Replay detection improves as attack delay increases, while short-delay attacks remain the most challenging cases.

Figure 4 summarizes replay detection across all experiments for different THV values. Increasing THV improves acceptance of legitimate packets (higher TP, lower FN), but also slightly increases acceptance of replayed packets (higher FP), reflecting the expected security-availability trade-off. Larger THV values allow replayed packets with small delays (i.e., $\Delta t \leq W$) to satisfy the timing-consistency condition. Overall, these results highlight the fundamental trade-off between robustness to natural timing variations and sensitivity to replay attacks, particularly for fast-relay or short-delay replays.

To further evaluate the effectiveness of SfS under different replay conditions, we analyze the detection trade-offs introduced by the timing acceptance window (THV). Figure 5 shows that replay detection improves as replay delay increases from 1–5 s. Short-delay replays remain more challenging because closely timed attacks are more likely to satisfy the receiver timing-consistency condition.

Figure 6 illustrates the core SfS trade-off between replay detection and legitimate-packet acceptance. Increasing THV improves TPR by reducing rejection of legitimate packets under timing variation, but also decreases TNR for short-delay replay attacks because replayed packets are more likely to fall within the acceptance window. Longer-delay replays remain reliably detected across all THV values. These results highlight the importance of selecting THV according to application timing and security requirements.

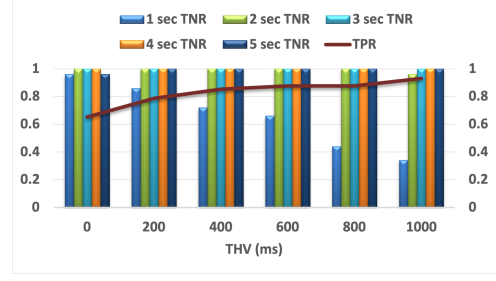


Figure 6: Trade-off between TNR and TPR across THV values. Larger THVs improve TPR but reduce replay sensitivity for short-delay attacks.

Different applications impose varying priorities between performance and security. Some systems prioritize acceptance of legitimate packets (high TPR), tolerating minor replay delays, while others require strict rejection of replayed packets (high TNR). To capture these requirements, we define a weighted cost function, where the weights reflect application-specific priorities:

$$J = w_{TP} \cdot TPR + w_{TN} \cdot TNR, \quad w_{TP} + w_{TN} = 1 \quad (5)$$

Figure 7 illustrates three weighting strategies: (1) *performance-critical* (higher weight on TPR), (2) *balanced*, and (3) *security-critical* (higher weight on TNR). The results show that increasing THV benefits performance-critical systems by improving TPR, leading to higher overall scores. In contrast, security-critical systems penalize larger THVs due to the reduction in TNR, particularly for short-delay replay attacks (e.g., 1-second). The balanced configuration achieves a compromise between these objectives but still reflects sensitivity to tightly timed replays at lower THV.

We further evaluate time-critical scenarios, where even millisecond-level delays may compromise the temporal relevance of the message. Figures 8 and 9 present replay attacks at *millisecond-scale delays*. Detection performance improves as replay delay increases from 100–900 ms, since larger delays are more likely to exceed the receiver acceptance window. In contrast, short-delay attacks remain more challenging, particularly at larger THV values. Application-specific weighting exhibits the same trend observed in the second-scale experiments. Performance-oriented configurations favor larger THVs to maximize legitimate-packet acceptance, whereas security-critical configurations prefer smaller THVs to maintain stronger replay rejection for short-delay attacks. Figure 10 illustrates the same fundamental SfS trade-off observed earlier: increasing THV improves acceptance of legitimate packets but reduces replay sensitivity, particularly for short-delay attacks (100–300 ms). For highly time-critical applications, smaller THV values therefore provide stronger replay resistance.

In our experimental setup, end-to-end latency remained on the order of a few milliseconds. Faster wireless technologies or optimized hardware may further reduce latency, enabling smaller THV values while preserving both replay detection and low false rejection rates.

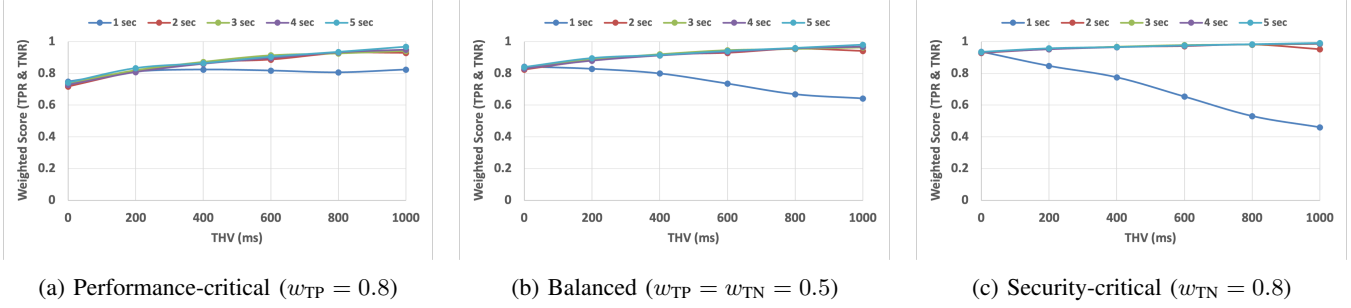


Figure 7: Impact of application-specific weighting on replay detection using the cost function in (5). Performance-oriented settings favor higher TPR, whereas security-oriented settings prioritize replay rejection (higher TNR).

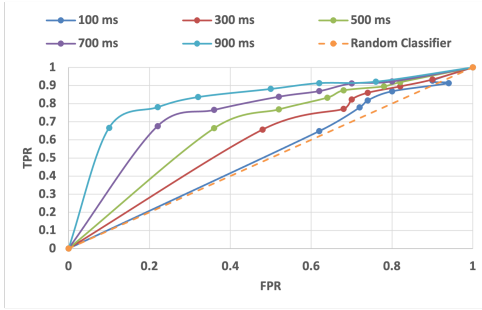


Figure 8: ROC curves for replay attacks delayed by 100–900 ms. Detection performance improves as replay delay increases, while short-delay attacks remain more challenging.

Permutation-aware Replay. Under a more capable threat model, we consider an *informed* adversary $\mathcal{A}$ who, after capturing a packet, attempts to re-order its bits/bytes so that the permutation matches what the receiver will expect at replay time, and then re-injects it. Two cases arise:

(i) *Same-epoch re-permutation* ($\Delta t \leq W$ and *epoch unchanged*). If the delay keeps the packet within the *same* seed/epoch e as the original send time (e.g., a millisecond-scale delay that lies inside the acceptance window W), then the expected permutation remains π_e . Any “re-permutation” targeting the current epoch simply reproduces the sender-signed byte order, so the signature **passes**, this is operationally indistinguishable from a within-window relay (Attack 2). If an application-level short-term replay cache is enabled, the duplicate would be **suppressed**. Moreover, because $\Delta t \leq W$ by construction, such small delays are explicitly accepted by the freshness policy and do not increase operational risk.

(ii) *Cross-epoch re-permutation* (Δt moves into a new epoch). If $\mathcal{A}$ delays long enough that a *new* seed/epoch $e' \neq e$ becomes active (e.g., seconds), the receiver expects $\pi_{e'}$. The sender’s signature commits to $\pi_e(m)$; without the sender’s key, $\mathcal{A}$ cannot produce a valid signature for $\pi_{e'}(m)$. Consequently, verification **fails** and the packet is **rejected**, regardless of W .

We evaluated replay attacks at both millisecond and second scales across multiple acceptance-window settings.

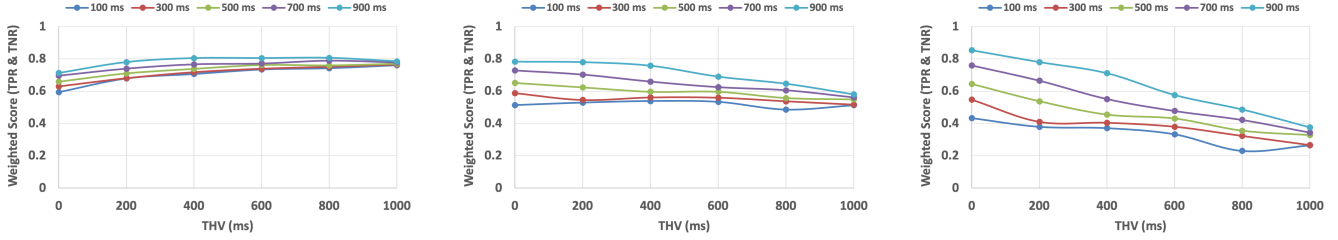
The results follow the predicted Δt – W frontier, as shown in Figure 11:

- *Seconds-scale replays (cross-epoch)*: For $\Delta t > 1$ s, TNR is ≈ 1 for all W (with only minor variance at $W=1000$ ms), confirming that cross-epoch re-permutations consistently **fail** signature verification.
- *Millisecond replays (same-epoch risk)*: The most challenging cases occur when Δt is close to W . Here, TNR decreases because the replay remains within the same epoch, cryptographic verification correctly **accepts** the signature, and discrimination relies on the policy choice of W .
- *Trade-off*: Increasing W improves availability for benign traffic (fewer false rejections in no-attack runs) but reduces TNR against near window replays, an intentional, operator-tunable trade-off inherent to SfS freshness schemes.

SfS *provably* rejects cross epoch replays via signature mismatch (seconds-scale delays) and behaves as expected for same-epoch replays (millisecond-scale delays): if the replay falls within the accepted freshness window W , cryptographic checks pass by design, and the deployment should rely on duplicate suppression and appropriate sizing of W relative to the beacon rate. In practice, setting W below the smallest harmful Δt for the application (e.g., below one beacon period) yields near-perfect rejection of adversarial replays while preserving availability.

5.3.3. Overhead and Throughput (Practicality). We evaluated the computational cost of our scheme on a Raspberry Pi 3B-based testbed, using one Pi as sender and a second Pi as receiver connected over Wi-Fi. The sender transmitted 1,000 authenticated packets back-to-back over UDP to stress the system, while the receiver verified each packet using the proposed time-based sequence reconstruction and signature check. Table 3 presents the performance summary of the SfS prototype implementation on Raspberry Pi 3B devices.

On the sender side, packet preparation (sequence generation from the time-based seed, bit shuffling, hashing, and RSA signing) costs on average $\approx 1.94 \times 10^5 \mu\text{s}$ per packet (≈ 194 ms). This corresponds to a theoretical throughput of roughly 5 packets/s, assuming the CPU is fully dedicated to signing. In our end-to-end experiment, the sender



(a) Performance-critical ($w_{TP} = 0.8$)

(b) Balanced ($w_{TP} = w_{TN} = 0.5$)

(c) Security-critical ($w_{TN} = 0.8$)

Figure 9: Impact of application-specific weighting on millisecond-scale replay detection using the cost function in (5).

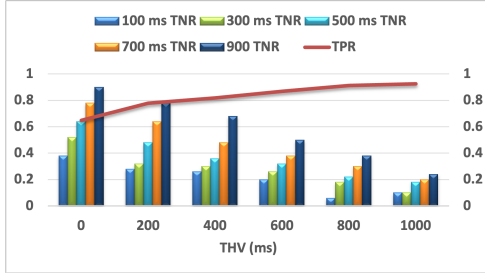


Figure 10: Trade-off between TNR and TPR. vs THVs for millisecond-scale replay. Larger THVs improve TPR but reduce replay sensitivity for short-delay attacks.

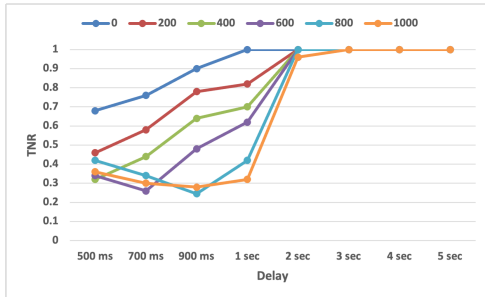


Figure 11: Permutation-aware Replay: TNR versus replay delay Δt for multiple THV values. Detection is most challenging near the acceptance-window boundary ($\Delta t \approx W$).

transmitted 1,000 packets in 196.24 s, yielding a measured sending rate of ≈ 5.10 packets/s, confirming that sender-side cryptographic processing is the dominant bottleneck. This overhead is dominated by software-based RSA on constrained hardware; adopting lightweight signatures (e.g., Ed25519) or hardware acceleration would significantly improve throughput.

The receiver side is significantly lighter. Reconstructing the seed, regenerating the sequence, unshuffling the bits, recomputing the hash, and verifying the RSA signature takes on average only $\approx 4.44 \times 10^3 \mu s$ per packet (≈ 4.4 ms), which allows a theoretical verification rate of about 225 packets/s. The receiver processed 1,000 packets in 224.28 s, corresponding to an effective throughput of ≈ 4.46 packets/s, which is limited by the sender's signing

TABLE 3: Performance summary of the prototype implementation on Raspberry Pi 3B.

Component	Metric	Value
Sender	Avg. time for signing+sequence+shuffle	194 ms
	Theoretical throughput	≈ 5 pkts/s
	Measured throughput (1,000 pkts)	5.10 pkts/s
Receiver	Avg. verification time	4.44 ms
	Theoretical throughput	≈ 225 pkts/s
	Measured throughput (1,000 pkts)	4.46 pkts/s
End-to-end	Avg. latency (sender $\rightarrow$ receiver)	401 ms
	95th percentile latency	1.17 s

rate rather than the verification cost. Measured end-to-end latency from sender timestamp to receiver processing averaged ≈ 401 ms, with a 95th percentile of ≈ 1.17 s, dominated by sender computation and wireless airtime. Given that our target applications use beacon rates on the order of 1–2 Hz, these results indicate that the proposed replay-detection mechanism is computationally feasible on low-cost embedded hardware.

6. Deployment Considerations and Protocol Integration (SfS Case Studies)

To demonstrate practical relevance beyond an abstract mechanism, we map SfS to representative wireless broadcast systems where authenticity is already deployed or actively emerging. These systems share common characteristics that make replay resilience important: periodic transmissions, open wireless channels, constrained packet formats, and time-sensitive operational information.

The considered systems represent different replay-protection maturity levels. Drone Remote ID systems increasingly adopt authenticated broadcasts but often lack dedicated replay protection mechanisms. V2X systems already employ timestamp-based freshness validation, making SfS attractive as a lightweight alternative that reduces explicit metadata overhead. In contrast, authenticated ADS-B and satellite telemetry proposals primarily address authenticity while replay resilience remains insufficiently explored, particularly under constrained payload sizes. SfS is therefore especially attractive for broadcast protocols where freshness protection must be introduced without packet expansion.

6.1. Drone Remote ID Broadcasts

Remote ID systems require unmanned aerial vehicles (UAVs) to periodically broadcast identity and operational state information over short-range wireless technologies such as Bluetooth or Wi-Fi [1]. These broadcasts are intended to support regulatory compliance, airspace awareness, and local identification of nearby drones. Typical transmissions occur at approximately **1 Hz** and may include information such as drone identity, location, altitude, velocity, and timestamp data [44].

Current Remote ID deployments vary in their security design [45]. Some implementations provide limited authentication support, while stronger cryptographic protection is expected to become increasingly important as drone traffic density and security requirements grow. Similar to V2X systems, explicit freshness metadata may consume valuable payload space and still rely on synchronized clocks.

Replay impact. Stale yet authentic broadcasts may create phantom drones, false restricted-zone violations, or misleading airspace activity.

How Sfs helps. Sfs implicitly encodes freshness through authenticated payload permutations rather than explicit freshness metadata, avoiding packet expansion while preserving lightweight operation suitable for constrained UAV platforms and dense drone environments.

6.2. Broader Application to Wireless Broadcast Systems

Vehicular Safety Beacons (V2X). Vehicular safety protocols such as CAM, DENM, and BSM continuously broadcast time-sensitive traffic information to nearby vehicles and roadside infrastructure [5], [46]. Current V2X security frameworks already employ signatures and timestamp-based freshness validation. Thus, current V2X systems are more resilient to replay attacks than many legacy broadcast systems. However, dedicated freshness fields consume payload space, increase metadata overhead.

Replay impact. Played safety beacons may trigger false hazards or misleading cooperative-driving decisions [47].

How Sfs helps. Sfs can complement timestamp-based freshness validation while reducing explicit metadata overhead in dense beaconing environments.

ADS-B Aviation Broadcasts. ADS-B periodically broadcasts aircraft identity and flight-state information using fixed 112-bit frames over open wireless channels [4]. Due to its open wireless nature and lack of native cryptographic protection, ADS-B has long been recognized as vulnerable to spoofing and replay attacks.

A large body of prior work has proposed adding cryptographic authentication (e.g., signatures) to mitigate spoofing and message forgery. However, many such approaches primarily address authenticity and integrity, while replay resilience remains insufficiently addressed. Even if a previously transmitted ADS-B packet is cryptographically valid,

TABLE 4: Representative broadcast systems where Sfs can provide replay resilience.

System	Auth.	TS	Replay Risk	Sfs Role
Drone RID	Partial	Limited	Phantom drones	Lightweight freshness
V2X	Yes	Yes	False hazards	Reduced metadata
ADS-B	Emerging	No	Ghost aircraft	No pkt. expansion
CCSDS	Yes	Often	Stale telemetry	Complement counters

an adversary may still retransmit it later to create stale yet believable air traffic information.

Replay impact. Stale yet authentic packets may create ghost aircraft, false trajectories, or misleading surveillance information [48].

How Sfs helps. Sfs can complement authenticated ADS-B proposals by embedding freshness into authenticated payload permutations without increasing packet size or modifying the fixed broadcast structure.

Satellite and Space Broadcast Systems. Satellite telemetry and navigation systems commonly rely on counters, timestamps, or delayed authentication mechanisms to maintain freshness guarantees [6], [49], [50]. These mechanisms are effective but may require persistent state synchronization between communicating entities, especially after long delays or intermittent connectivity.

Replay impact. Replay attacks may introduce stale telemetry, delayed navigation data, or misleading spacecraft state information.

How Sfs helps. Sfs can complement these mechanisms by embedding transmission-time semantics directly into authenticated payload structure without introducing additional freshness fields or persistent replay state.

Across aviation, drones, vehicular networks, and satellite systems, cryptographic authentication primarily verifies *who sent the packet*, while freshness determines *whether the information is still valid*. Sfs addresses this second challenge by binding transmission-time semantics directly to authenticated payload structure, enabling replay resilience without increasing packet size.

7. Conclusion

This paper presented *Shuffling for Safeguarding (Sfs)*, a lightweight replay protection mechanism for signed wireless broadcasts that provides implicit freshness without introducing additional protocol fields or explicit timestamp payloads. Sfs binds transmission time to authenticated payload structure through deterministic time-derived permutations, enabling replay detection while remaining compatible with constrained broadcast protocols. Experimental evaluation on a Raspberry Pi IEEE 802.11 testbed demonstrated strong replay-detection performance under both second-scale and millisecond-scale delays, while preserving robust acceptance of legitimate packets under configurable timing tolerances. The results further highlighted the trade-off between replay sensitivity and timing robustness governed by the THV

parameter. SfS is particularly attractive for constrained wireless broadcast systems such as Drone Remote ID, ADS-B, V2X, and satellite telemetry, where replay resilience must be achieved without packet expansion. Future work will explore adaptive THV selection and broader integration with authenticated wireless communication protocols.

References

- [1] P. Tedeschi, S. G. Ganti, and S. Sciancalepore, "Selective authenticated pilot location disclosure for remote id-enabled drones," *Proceedings on Privacy Enhancing Technologies*, vol. 2024, no. 3, pp. 523–539, 2024.
- [2] O. M. Moushi, C. Gunawardena, F. Ye, R. Q. Hu, and Y. Qian, "Machine learning-based detection of data replay and data replay sybil attacks for vehicular communication networks," in *ICC 2024 - IEEE International Conference on Communications*, 2024, pp. 5202–5207.
- [3] T. Kacem and S. Tossou, "Ads-b replay attack detection using transformers," in *2024 International Wireless Communications and Mobile Computing (IWCMC)*, 2024, pp. 144–149.
- [4] H. A. Khan, H. Khan, S. Ghafoor, and M. A. Khan, "A survey on security of automatic dependent surveillance-broadcast (ads-b) protocol: Challenges, potential solutions, and future directions," *IEEE Communications Surveys & Tutorials*, 2024.
- [5] E. L. Cominetti, M. V. M. Silva, M. A. Simplicio Jr, H. K. Patil, and J. E. Ricardini, "Faster verification of v2x bsm messages via message chaining," *Cryptology ePrint Archive*, 2022.
- [6] S. Shruthi, P. Pragathi, N. Shwetha, and S. Swathi, "Ccsds packet telemetry," in *2017 IEEE International Conference on Smart Technologies and Management for Computing, Communication, Controls, Energy and Materials (ICSTM)*. IEEE, 2017, pp. 371–376.
- [7] M. Singh, P. Leu, A. Abdou, and S. Capkun, "Uwb-ed: distance enlargement attack detection in ultra-wideband," in *Proceedings of the 28th USENIX Conference on Security Symposium*, ser. SEC'19. USA: USENIX Association, 2019, p. 73–88.
- [8] P. Leu, M. Singh, M. Roeschlin, K. G. Paterson, and S. Čapkun, "Message time of arrival codes: A fundamental primitive for secure distance measurement," in *2020 IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 500–516.
- [9] T. Aura, "Strategies against replay attacks," in *Proceedings 10th Computer Security Foundations Workshop*, 1997, pp. 59–68.
- [10] S. Chandrasekaran, K. Ramachandran, S. Adarsh, and A. K. Puranik, "Avoidance of replay attack in can protocol using authenticated encryption," in *2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, 2020, pp. 1–6.
- [11] F. Farha, H. Ning, S. Yang, J. Xu, W. Zhang, and K.-K. R. Choo, "Timestamp scheme to mitigate replay attacks in secure zigbee networks," *IEEE Transactions on Mobile Computing*, vol. 21, no. 1, pp. 342–351, 2022.
- [12] M. Schäfer, M. Strohmeier, V. Lenders, I. Martinovic, and M. Wilhelm, "Bringing up opensky: A large-scale ads-b sensor network for research," in *IPSN-14 Proceedings of the 13th International Symposium on Information Processing in Sensor Networks*, 2014, pp. 83–94.
- [13] B. Ying and A. Nayak, "Anonymous and lightweight authentication for secure vehicular networks," *IEEE Transactions on Vehicular Technology*, vol. 66, no. 12, pp. 10626–10636, 2017.
- [14] M. Hamadouche, Z. Khalil, H. Tebbi, M. Guerroumi, and Y. Zafoune, "A replay attack detection scheme based on perceptual image hashing," *Multimedia Tools and Applications*, vol. 83, no. 3, pp. 8999–9031, 2024.
- [15] Civil Aviation Safety Authority (CASA), "How ads-b works," n.d., accessed: 2025-07-28. [Online]. Available: <https://www.casa.gov.au/operations-safety-and-travel/airspace/automatic-dependent-surveillance-broadcast-ads-b/how-ads-b-works>
- [16] R. Al-ani, B. Zhou, Q. Shi, T. Baker, and M. Abdhamed, "Adjusted location privacy scheme for vanet safety applications," in *NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium*, 2020, pp. 1–4.
- [17] T. Kos, I. Markezic, and J. Pokrajcic, "Effects of multipath reception on gps positioning performance," in *Proceedings ELMAR-2010*, 2010, pp. 399–402.
- [18] Furuno Electric Co., Ltd., "What is gps?" n.d., accessed: 2025-07-28. [Online]. Available: https://www.furuno.com/en/gnss/technical/tec_what_gps
- [19] K. Zheng, Q. Hu, and J. Zhang, "Positioning error analysis of ranging-mode using ais signals in china," *Journal of Sensors*, vol. 2016, no. 1, p. 6928961, 2016. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1155/2016/6928961>
- [20] U.S. Coast Guard Navigation Center, "Ais frequently asked questions," n.d., accessed: 2025-07-28. [Online]. Available: <https://www.navcen.uscg.gov/ais-frequently-asked-questions>
- [21] J. D. Ger and M. K. and, "Assessing real time gps asset tracking for timber haulage," *The Open Transportation Journal*, vol. 3, 2009/8/27. [Online]. Available: <https://opentransportationjournal.com/abs/10.2174/1874447800903010078>
- [22] Hughes Systique Corporation, "Big data for location tracking," n.d., accessed: 2025-07-28. [Online]. Available: <https://www.hsc.com/resources/blog/big-data-for-location-tracking>
- [23] H. Gilbert and H. Handschuh, "Security analysis of sha-256 and sisters," in *Selected Areas in Cryptography*, M. Matsui and R. J. Zuccherato, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 175–193.
- [24] A. Costin and A. Francillon, "Ghost in the air (traffic): On insecurity of ADS-B protocol and practical attacks on ADS-B devices," *Blackhat USA*, vol. 1, pp. 1–12, 2012.
- [25] M. Lenhart, M. Spanghero, and P. Papadimitratos, "Relay/replay attacks on gnss signals," in *Proceedings of the 14th ACM Conference on Security and Privacy in Wireless and Mobile Networks*, ser. WiSec '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 380–382. [Online]. Available: <https://doi.org/10.1145/3448300.3468256>
- [26] F. Farha, H. Ning, S. Yang, J. Xu, W. Zhang, and K.-K. R. Choo, "Timestamp scheme to mitigate replay attacks in secure zigbee networks," *IEEE Transactions on Mobile Computing*, vol. 21, no. 1, pp. 342–351, 2020.
- [27] C. Cremers, B. Kiesl, and N. Medinger, "A formal analysis of IEEE 802.11's WPA2: Countering the cracks caused by cracking the counters," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 1–17. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity20/presentation/cremers>
- [28] M. Braunschmied, "Countering suci-catcher-practical implementation, evaluation and mitigation against suci replay attacks in 5g networks," Ph.D. dissertation, Technische Universität Wien, 2025.
- [29] K.-F. Krentz and T. Voigt, "More lightweight, yet stronger: Revisiting oscore's replay protection," in *Proc. of the NDSS Workshop on Security and Privacy in Standardized IoT (SDIoTSec)*. Internet Society, 2024.
- [30] A. Perrig, R. Canetti, J. Tygar, and D. Song, "Efficient authentication and signing of multicast streams over lossy channels," *02 2000*, pp. 56–73.
- [31] A. Perrig, D. Song, R. Canetti, J. Tygar, and B. Briscoe, "Timed efficient stream loss-tolerant authentication (tesla): Multicast source authentication transform introduction," *Tech. Rep.*, 2005.

- [32] J. Anderson, S. Lo, and T. Walter, "Time synchronization of tesla-enabled gnss receivers," *IEEE Transactions on Aerospace and Electronic Systems*, 2025.
- [33] W. Fan, B. Shi, and C. Peng, "Nreplay: 5g key reinstallation attack based on nas layer vulnerabilities," in *MILCOM 2024-2024 IEEE Military Communications Conference (MILCOM)*. IEEE, 2024, pp. 1088–1093.
- [34] F. Tang, L. Wu, and J. Liao, "Review of radio frequency fingerprint technology applied in satellite security protection," in *2024 4th International Conference on Computer Science, Electronic Information Engineering and Intelligent Control Technology (CEI)*. IEEE, 2024, pp. 257–262.
- [35] N. Basha, B. Hamdaoui, A. Erbad, and M. Guizani, "On the detection of replay authentication attacks through channel state information analysis," in *GLOBECOM 2024-2024 IEEE Global Communications Conference*. IEEE, 2024, pp. 1719–1724.
- [36] R. Abubaker and G. Gong, "Relay attack detection using ofdm channel-fingerprinting," *Authorea Preprints*, 2024.
- [37] K. Jansen, L. Niu, N. Xue, I. Martinovic, and C. Pöpper, "Trust the crowd: Wireless witnessing to detect attacks on ads-b-based air-traffic surveillance," in *NDSS*, 2021.
- [38] G. Ray and J. Ray, "Detecting ads-b replay cyberattacks in the national airspace system," *Issues in Information Systems*, vol. 24, no. 1, 2023.
- [39] S. Gueron, S. Johnson, and J. Walker, "Sha-512/256," in *2011 Eighth International Conference on Information Technology: New Generations*, 2011, pp. 354–358.
- [40] Raspberry Pi Foundation, "Raspberry pi 4 model b," <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/>, 2019, accessed: 2025-08-05.
- [41] T.-H. Lai and D. Zhou, "Efficient, and scalable ieee 802.11 ad-hoc mode timing synchronization function," in *17th International Conference on Advanced Information Networking and Applications, 2003. AINA 2003.*, 2003, pp. 318–323.
- [42] E. Milanov, "The rsa algorithm," *RSA laboratories*, vol. 1, no. 11, 2009.
- [43] Chrony Project, "Chrony: Versatile implementation of the network time protocol (ntp)," <https://chrony-project.org/>, 2025, accessed: 2025-08-05.
- [44] E. M. Wisse, "A2rid: An anonymous remoteid-compliant group signature scheme for commercial drones," Master's thesis, Eindhoven University of Technology, 2022.
- [45] S. Khan, G. S. Gaba, F. Boeira, and A. Gurtov, "Formal verification and security assessment of the drone remote identification protocol," in *Linköping University Research Archive*, 2024.
- [46] J.-P. Monteuiis, J. Petit, C. Chen, S. Das, M. Nekoui, and S. Yang, "V2x misbehavior in decentralized notification basic service: Considerations for standardization," in *Proceedings of the 2024 Cyber Security in CarS Workshop (CSCS '24)*. ACM, 2024.
- [47] M. Y. Muhammad, "Efficient authentication of vehicle-to-vehicle (v2v) safety messages," Ph.D. dissertation, Birmingham City University, 2024.
- [48] W. Ahmed, A. Masood, and J. Manzoor, "Towards secure air traffic surveillance: A survey on ads-b threats, existing solutions, and future research," *IEEE Open Journal of the Computer Society*, 2026.
- [49] A. Savchenko, "Authenticated encryption for space telemetry," in *76th International Astronautical Congress (IAC 2025)*, Sydney, Australia, 2025.
- [50] J. Willbold, F. Sciberras, M. Strohmeier, and V. Lenders, "Satellite cybersecurity reconnaissance: Strategies and their real-world evaluation," in *IEEE Aerospace Conference*. IEEE, 2024.
- [51] "Plantuml at a glance," 2025, accessed: 2025-08-05. [Online]. Available: <https://www.plantuml.com/plantuml/uml/SyfFKj2rKt3CoKnELR1Io4ZDoSa70000>
- [52] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*. Chapman and Hall/CRC, 2007.

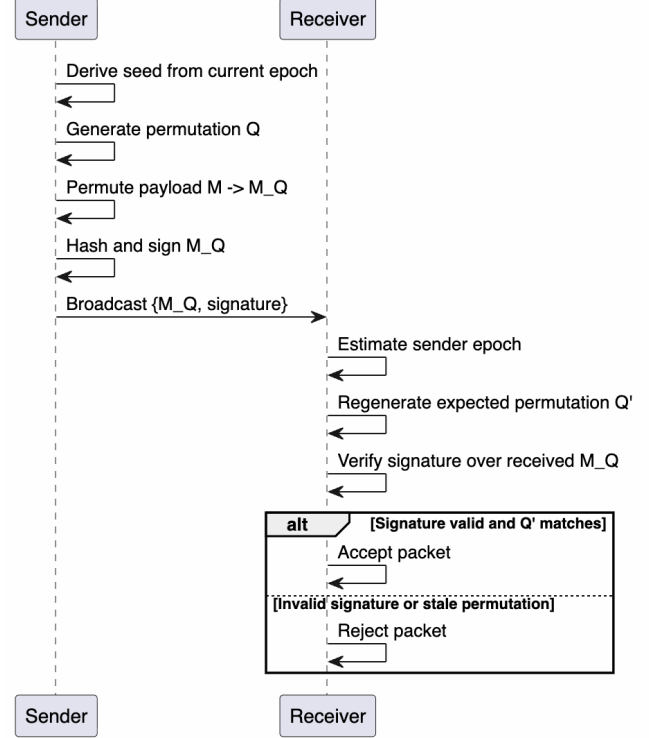


Figure 12: Sequence diagram of the proposed secure communication scheme, created by the PlantUML tool [51]. The sender computes a hash of the message concatenated with a generated bit sequence, signs it, and broadcasts the message with the signature. The receiver reconstructs the expected sequence, verifies the signature, and validates message integrity before accepting or rejecting the packet.

Appendix A. SfS Operational Workflow

This appendix provides a detailed sequence diagram of the end-to-end SfS workflow. The diagram summarizes the sender and receiver side operations involved in seed derivation, payload permutation, signature verification, and replay detection.

Figure 12 presents the full sequence diagram of the proposed secure communication scheme, summarizing the interactions between the sender and receiver from bit sequence generation and signing to message verification and integrity validation.

Appendix B. Mathematical Model and Security Proofs (Extended Formalization)

Let S (Sender) and R (Receiver) be two entities communicating over a potentially insecure wireless channel. The channel transmits messages consisting of n bits in a predefined sequence q . To ensure message integrity, authenticity,

and non-repudiation, the communication system employs a digital signature scheme (Sign, Verify), which signs the message digest $H(m)$, where H is a hash function and m is the message. This section formalizes the system and provides theoretical guarantees under standard cryptographic assumptions [52].

B.1. Model Properties

- **Payload Space:** Define the set of all possible plaintext messages (payloads) as $M = \{m \mid m \text{ is a valid plaintext}\}$. Each message includes meta-data such as sender location, speed, and any additional required information.
- **Bit Sequences:** Define the set of permutations Q , where each Q is a structured arrangement of n indices such that the payload m is transmitted according to Q . The length of Q is denoted as $n = |m|$.
- **Hash Function:** The function $H : M \rightarrow \mathcal{H}$ maps payloads to a hash space $\mathcal{H}$, satisfying:
 - *Pre-image resistance:* For any $h \in \mathcal{H}$, it is computationally infeasible to find $m \in M$ such that $H(m) = h$.
 - *Collision resistance:* It is computationally infeasible to find $m_1, m_2 \in M$, $m_1 \neq m_2$, such that $H(m_1) = H(m_2)$.
- **Digital Signature Scheme:**
 - Each entity X has a key pair (PK_X, SK_X) , where PK_X is the public key and SK_X is the private key.
 - The signature function $\text{Sign} : \mathcal{H} \times SK \rightarrow \Sigma$ maps a hash value and private key to a digital signature.
 - The verification function $\text{Verify} : \mathcal{H} \times \Sigma \times PK \rightarrow \{0, 1\}$ checks the validity of the signature.
- **Communication Process:**
 - **Sender:** Given a message $m \in M$ of n bits, at time t_{sent} , the sender computes:

$$\begin{aligned} S &= f(t_{\text{sent}}), \quad q = \text{Perm}(S), \\ m_Q &= \text{Permute}(m, q), \quad h = H(m_Q), \\ \sigma &= \text{Sign}(h, SK_S), \end{aligned}$$

and transmits (m_Q, σ) according to the sequence defined in q .

- **Receiver:** Upon receiving (m_Q, σ) , the receiver computes $h' = H(m_Q)$ as per the protocol and verifies the integrity of the message:

$$\text{Verify}(h', \sigma, PK_S) = 1.$$

If the verification succeeds, m is accepted; otherwise, it is rejected.

B.2. Security Definitions

The security guarantees of this protocol are defined in terms of resistance against tampering, forgery, and repudiation [52].

Definition 1 (Message Integrity – Resistance to Tampering). A secure communication system is α -resistant to tampering if for any adversarial modification $m'_Q \neq m_Q$, the probability of the receiver accepting the message remains at most α :

$$\Pr_A[\text{Verify}(H(m'_Q), \sigma, PK_S) = 1] \leq \alpha$$

Definition 2 (Authenticity – Resistance to Forgery). A digital signature scheme is β -resistant to forgery if an adversary cannot produce a valid signature for an arbitrary message m'_Q without knowing SK_S :

$$\Pr_A[\exists m'_Q : \text{Verify}(H(m'_Q), \sigma', PK_S) = 1] \leq \beta$$

Definition 3 (Non-Repudiation – Resistance to Denial).

A digital signature scheme ensures γ -resistance to repudiation if the sender cannot deny signing a message m_Q once (m_Q, σ) has been transmitted and verified:

$$\Pr_S[\text{Deny}(m_Q, \sigma) \mid \text{Verify}(H(m_Q), \sigma, PK_S) = 1] \leq \gamma$$

Definition 4 (Replay Resistance).

A system is δ -resistant to replay if an adversary cannot successfully replay a previously transmitted message (m_Q, σ) at a later time $t_2 \neq t_1$ unless the permutation regenerated at t_2 matches the original permutation used at t_1 :

$$\Pr_A[P(t_2) = P(t_1)] \leq \delta$$

where $P(t)$ denotes the permutation derived from the time-dependent seed at time t .

B.3. Security Proofs

Lemma 1: Message Integrity If $H(m_Q) \neq H(m'_Q)$ for $m_Q \neq m'_Q$, then tampering is detectable.

Proof: The collision-resistance property of H ensures that any alteration of the transmitted message results in $H(m'_Q) \neq H(m_Q)$. Thus, the receiver can detect tampering by verifying the signature.

Lemma 2: Authenticity and Non-Repudiation The signature $\sigma = \text{Sign}(H(m_Q), SK_S)$ uniquely authenticates the sender and prevents forgery.

Proof: The security of Sign guarantees that only S (who holds SK_S) can generate σ . Verification with PK_S ensures authenticity. Non-repudiation follows because S cannot deny generating σ .

Lemma 3: Replay Resistance

A delayed replay of a previously transmitted message succeeds only if the permutation generated at the replay time matches the original permutation:

$$P(t_2) = P(t_1)$$

Proof: Assume a message was originally transmitted at time t_1 using permutation $P(t_1)$. If an adversary replays the same transmitted pair (m_Q, σ) at a later time t_2 , the receiver reconstructs the expected permutation from the estimated sending time. If $P(t_2) \neq P(t_1)$, the reconstructed ordering is inconsistent with the received permuted message, and the replay is rejected. Therefore, a successful delayed replay requires permutation recurrence.

Lemma 4: End-to-End Security If H and (Sign, Verify) are secure, the system guarantees message integrity, authenticity, and non-repudiation.

Proof: Combining Lemmas 1 and 2, any alteration of m_Q is detectable, and only the legitimate sender can produce a valid σ . Hence, the system is secure against forgery and tampering.

At the end, the proposed communication model ensures secure transmission over an insecure wireless channel using a combination of a collision-resistant hash function and a secure digital signature scheme. The model guarantees the following:

- **Message Integrity:** Tampering with m_Q is detectable via $H(m_Q)$.
- **Authenticity:** Only the sender S can produce valid signatures using SK_S .
- **Non-repudiation:** The sender cannot deny generating σ .
- **Replay Resistance:** A delayed replay is accepted only if the future permutation matches the original permutation, which is negligible for realistic payload sizes.

This formalization provides a robust foundation for practical implementations of secure communication protocols in systems requiring integrity and authenticity guarantees.

Appendix C. SfS Artifact

The artifact for *Shuffling for Safeguarding (SfS): Replay Protection in Wireless Broadcast Systems* contains (i) the packet-level experimental workbooks and analysis program needed to regenerate Figures 3–11 and (ii) the parameterized Python programs and documentation for the three-device wireless testbed. It supports the paper’s legitimate-traffic, ordinary-replay, permutation-aware replay, and prototype-performance claims. We request the *Available* and *Reproduced* badges.

C.1. Access and Public Release

The public development repository is available at <https://github.com/afd1479/SfS-Artifact>. The permanent archival snapshot is available at <https://doi.org/10.5281/zenodo.22801028>. The archived record contains the complete evaluated code, data, scripts, and documentation. No proprietary software, paid service, private dataset, account, API key, or production credential is required. The artifact contains no analytics or evaluator tracking.

Source code and software scripts are released under the MIT License. The experimental datasets, generated tables and figures, and documentation are released under the Creative Commons Attribution 4.0 International License (CC BY 4.0). The repository root contains the authoritative license files, citation metadata, installation entry point, and evaluator README.

C.2. Artifact Contents and Claims

The principal repository components are:

- `SfS_Artifact_Reproduction/`: raw .xlsx workbooks, `reproduce_all.py`, the plot-to-data mapping, derived CSV tables, and generated PNG/PDF figures;
- `SfS_RaspberryPi_Testbed_v1.0/`: sender, receiver, ordinary replay attacker, permutation-aware replay attacker, performance programs, permutation analysis, and testbed documentation;
- `claims/`: a claim-to-artifact map with expected evidence and an executable entry point for the offline result reproduction;
- `install.sh`: isolated installation profiles for the offline analysis, Raspberry Pi programs, or both; and
- `HARDWARE_ACCESS.md`: the physical requirements, calibration procedure, remote-access limitation, and alternative evaluation paths.

C.3. Requirements and Hardware Access

Offline reproduction.. This is the primary evaluation path. It runs on a commodity workstation with Python 3.10 or newer and `matplotlib`>=3.7. No GPU, privileged

access, special hardware, network service, or graphical interface is required. Once the dependency is installed, the complete Figure 3–11 workflow normally finishes in less than one minute.

Fresh over-the-air evaluation.. A live run requires three Raspberry Pi devices with 2.4 GHz Wi-Fi interfaces and drivers supporting IEEE 802.11 IBSS/ad-hoc mode, power supplies and microSD cards, and a controlled line-of-sight test area. The authors used approximately 3–5 m separation. The reference software environment uses Raspberry Pi OS, Python 3, NetworkManager, `iw`, `wireless-tools`, and Chrony. Administrative access is needed for the initial wireless and clock configuration.

The radio experiment cannot run faithfully on an ordinary cloud VM because it depends on three physical IBSS nodes and their over-the-air timing. The authors do not provide an always-on Internet-accessible SSH testbed. Although SSH itself is lightweight, stable remote evaluation would require secure management access to three devices, separation of management and experimental traffic, concurrent process coordination, and physical recovery when a radio or device becomes unreachable. Management activity can also perturb the performance measurements. The hardware-free workflow therefore serves as the primary evaluator path. Evaluators with equivalent hardware may perform the supplied scaled or full live procedures. If direct access is judged essential, feasibility must first be discussed through the official artifact review channel as described in `HARDWARE_ACCESS.md`.

C.4. Evaluation Workflow A: Regenerate Figures 3–11

From the repository root, run:

```
bash install.sh analysis
bash claims/claim1_figures_3_11/run.sh
```

The installer creates `.venv-analysis`. The claim runner enters `SfS_Artifact_Reproduction/` and invokes `reproduce_all.py` without modifying the raw workbooks under `data/`. A successful run exits with status zero and produces:

- PNG and PDF versions of Figures 3–11 under `figures/`;
- the numerical CSV inputs to every plot under `derived/`; and
- `derived/validation_report.csv` with packet-count checks.

`PLOT_MAPPING.md` maps every generated file to its paper figure. Minor visual differences across Matplotlib versions are acceptable; the derived counts and rates are the primary comparison values. As a corrected Figure 11 spot check, the permutation-aware workbook for THV = 800 ms and replay delay = 900 ms contains 1,000 legitimate and 50 replay observations. Its derived replay counts are TN= 21 and FP= 29, giving TNR = 21/50 = 0.42.

TABLE 5: Mapping from the main paper results to artifact evidence.

Paper result	Artifact component	Evaluation and expected evidence
Figure 3	data/no_attack/	Regenerate the no-attack acceptance counts from 1,000 legitimate packets at each THV.
Figures 4–7	data/replay_seconds/	Recompute aggregate confusion-matrix counts, ROC points, TPR/TNR trade-offs, and weighted scores for replay delays of 1–5 s.
Figures 8–10	data/replay_milliseconds/	Recompute ROC points, weighted scores, and TPR/TNR trade-offs for replay delays of 100–900 ms.
Figure 11	data/permutation_aware/ smart_replay_attacker.py	and Recompute the rejection rate of permutation-aware replay across the supplied THVs and replay delays; optionally exercise the live modified-replay path.
Ordinary-replay claim	ordinary_replay_attacker.py, receiver code, and Claims 1–2	Inspect the complete data and optionally confirm delayed, unmodified replay behavior with Scaled Runs B–C.
Prototype performance	sender_throughput.py, receiver_throughput.py, and Claim 4	Run the instrumented sender/receiver path and preserve its workbooks and terminal summaries; numerical results are environment-dependent.

Running the workflow again overwrites only the generated figures/ and derived/ contents. The raw data/ workbooks are not modified.

C.5. Evaluation Workflow B: Scaled Wireless Testbed

The authoritative setup and run instructions are in:

- docs/RASPBERRY_PI_SETUP.md;
- docs/PARAMETERS.md;
- docs/SCALED_EVALUATION.md;
- docs/RUN_EXPERIMENTS.md; and
- docs/OUTPUTS_AND_PERFORMANCE.md.

The default roles are sender (Pi 1, 192.168.1.11), receiver (Pi 2, 192.168.1.12), and attacker (Pi 3, 192.168.1.10), using UDP port 5005. The programs expose the addresses, port, packet count, THV, replay delay, replay interval, output filename, and other experimental settings through command-line options. Place all devices on the same isolated IBSS network, verify connectivity, and confirm Chrony synchronization. Always start the receiver first, the selected attacker second, and the sender last. The two attacker programs must not run simultaneously.

The scaled evaluation uses the same programs as the full experiment with 100 sender packets per run:

- 1) **Run A – calibration and performance:** execute `sender_throughput.py` and `receiver_throughput.py` without an active attacker, save both workbooks, and calculate the local mean observed delay from the receiver workbook.
- 2) **Run B – legitimate traffic:** execute `sender.py` and `receiver.py` without an attacker using the calibrated delay. The programs must create the documented workbooks; the exact delivery and verification rates are environment-dependent.
- 3) **Run C – ordinary replay:** add `ordinary_replay_attacker.py` with a 3 s delay and `--replay-every20`. Up to five packets are selected when all sender datagrams reach the attacker. Delivered

replays appear as repeated packet identifiers and are expected to fail at THV = 1000 ms.

- 4) **Run D – permutation-aware replay:** replace the ordinary attacker with `smart_replay_attacker.py`, retaining the 3 s delay and replay interval. Delivered modified replays carry the `smart_replay` label and are expected to fail at THV = 1000 ms.

The four scaled runs normally take approximately 15–30 minutes after the three-device testbed is configured. They verify execution and evidence generation, not the paper’s complete statistical rates. The full 1,000-packet and THV/delay procedures are documented in `RUN_EXPERIMENTS.md`. Evaluators should use a distinct `--output` filename for every run because reusing a path overwrites the previous workbook.

C.6. Calibration, Coordinates, and Interpretation

The laboratory did not use live GPS/GNSS devices. Sender and receiver coordinates are configurable experimental metadata; changing only the coordinates does not change verification timing in the current implementation. The active receiver estimate uses a calibrated mean end-to-end delay. The authors measured 0.29 s on their testbed, but this value includes the behavior of that complete environment and is not a universal physical propagation time. A new testbed must obtain its own value from Scaled Run A and pass it with `--mean-delay-s`. A future deployment may obtain accurate positions from GPS/GNSS, but a coordinate-derived timing method must be implemented and validated consistently at the receiver.

Exact live throughput, latency, packet delivery, and verification counts may vary with Raspberry Pi model, operating system, Python version, CPU load, storage, placement, interference, packet loss, and synchronization. Such values are not pass/fail targets for the scaled run. The deterministic Figure 3–11 workflow is the quantitative verification of the supplied paper results.

C.7. Safety, Provenance, and Ethics

The live programs use Python `pickle` serialization and include an experimental RSA key pair. They must be used only on a controlled, isolated research network with trusted devices; neither mechanism is intended for production deployment. The released workbooks were generated on the authors' controlled wireless testbed at New York University Abu Dhabi. They contain experiment packet identifiers, timings, classifications, and performance measurements, but no human-subject data, personal data, production user traffic, or captured credentials.